CS 410/510 · NORMALIZATION · WALKTHROUGH

First Normal Form

The registrar hands you a spreadsheet. It answers every question anyone has ever asked of it. It is also not a relation.

We are going to take one real-looking table apart, slowly, and name every reason it fails the weakest normal form there is. Then we will rebuild it three different ways and argue about which rebuild is right.

> **WHY THIS MATTERS**
>
> 1NF is the easiest normal form to satisfy and the easiest to violate accidentally, because spreadsheets encourage exactly the habits it forbids. Everything in Chapter 14 assumes you have already cleared it.

SLIDE 2 · THE RELATION AS GIVEN

STUDENT_ADVISING

```
STUDENT_ADVISING(StudentID, StudentName, Address, Phones, Courses)
```

StudentID	StudentName	Address	Phones	Courses
S1042	Marisol Vega	1910 University Dr, Boise, ID 83725	{208-426-1000, 208-426-5555}	{(CS 410, Fall 2026, A), (CS 321, Spring 2026, B)}
S1198	Theo Blackfoot	455 Lusk St, Boise, ID 83706	{208-426-3317}	{(CS 410, Fall 2026, B)}

One row per student. Every fact about a student is right there, no joins required. This is what a well-meaning spreadsheet looks like.

Question for the room: how many distinct things are wrong with this, and are they the same kind of wrong?

SLIDE 3 · STEP 1 — NAME THE VIOLATIONS

Three failures, three kinds

1NF requires every attribute value to be **atomic** and **single-valued**. Three attributes break that, and they break it differently.

1. **Address is composite.** Street, city, state, and zip packed into one string. You cannot ask for every student in Boise without parsing text.
2. **Phones is multivalued.** A set in a single cell, and the number of elements varies row to row.
3. **Courses is a nested relation.** Worse still — each element is itself a tuple with its own internal structure.

> THE DISTINCTION IS NOT PEDANTRY
>
> Composite gets fixed by splitting columns. Multivalued and nested need a new relation. Conflate them and you will either over-decompose an address or under-decompose a course list.

SLIDE 4 · STEP 2 — THE REMEDIES

Three ways to fix a multivalued attribute

Elmasri and Navathe give three. All three are legal. Only one of them is right here, and the reason why is the actual content of this walkthrough.

- **R1** — expand the key and flatten.
- **R2** — move the attribute into its own relation.
- **R3** — replace it with a fixed number of atomic columns.

> **HOW TO CHOOSE**
>
> Ask what each one asserts about the mini-world that the mini-world does not actually say. The wrong remedy always invents a fact.

SLIDE 5 · REMEDY R1

Expand the key

Repeat the other attributes until every cell is atomic. The key becomes {StudentID, Phone, CourseNum, Term}.

StudentID	StudentName	Phone	CourseNum	Term	Grade
S1042	Marisol Vega	208-426-1000 ◆	CS 410 ◆	Fall 2026	A
S1042	Marisol Vega	208-426-1000 ◆	CS 321 ◆	Spring 2026	B
S1042	Marisol Vega	208-426-5555 ◆	CS 410 ◆	Fall 2026	A
S1042	Marisol Vega	208-426-5555 ◆	CS 321 ◆	Spring 2026	B

Cells marked with ◆ are the ones being multiplied: two phones × two courses = four rows.

Every cell is now atomic. The relation is, strictly, in 1NF.

SLIDE 6 · WHY R1 IS WRONG

It invents a fact

Row three asserts a connection between the phone number 208-426-5555 and the course CS 410. **There is no such connection.** Marisol's phones and Marisol's courses are independent facts that happen to share a student.

- The row count is a **cross product**, not data. Four phones and five courses would give twenty rows for one student.
- Her grade in CS 410 is now stored twice. Change it once and the database contradicts itself.
- Deleting a phone number deletes course enrollments.

> **LOOKING AHEAD**
>
> If this smells like something with a name, it is. Two independent multivalued facts in one relation is a *multivalued dependency* — we get to it later today under 4NF. For now the lesson is simpler: never flatten two independent lists into one table.

SLIDE 7 · REMEDY R3

Fixed columns — the repeating group

If we know a student has at most three phones, replace the attribute with `Phone1, Phone2, Phone3`.

```
STUDENT(StudentID, StudentName, Address, Phone1, Phone2, Phone3, ...)
```

Every cell atomic. Technically 1NF. And still a bad design:

- A fourth phone requires altering the table — the schema encodes a limit that is not a real rule.
- Theo has one phone and carries two NULLs.
- "Which student has 208-426-5555?" becomes three equality tests joined by OR.

> **REMEMBER THIS ONE**
>
> A repeating group passes 1NF and fails common sense. Normal forms are a floor, not a ceiling. You will meet this pattern again in about twenty minutes.

SLIDE 8 · REMEDY R2

One relation per multivalued fact

```
STUDENT(StudentID, StudentName, Street, City, State, Zip)
    PK  StudentID

STUDENT_PHONE(StudentID, Phone)
    PK  {StudentID, Phone}
    FK  StudentID → STUDENT

ENROLLMENT(StudentID, CourseNum, Term, Grade)
    PK  {StudentID, CourseNum, Term}
    FK  StudentID → STUDENT
```

- **Address** was composite, so it became columns — no new relation needed.
- **Phones** and **Courses** were independent multivalued facts, so each got its own relation. Two facts, two relations.
- **Term** is in the ENROLLMENT key because a student may retake a course.

SLIDE 9 · WHAT IT BOUGHT US

The anomalies are now impossible

BEFORE AND AFTER

- **No cross product.** Marisol is one STUDENT row, two STUDENT_PHONE rows, two ENROLLMENT rows — five rows holding five facts, instead of four rows holding four copies of two facts.
- **Grades stored once.** No way to contradict yourself.
- **Phones and enrollments move independently.** Dropping a phone cannot touch a transcript.
- **No arbitrary cap.** Six phones, one phone, none at all — all representable without schema changes.

The cost is joins. That is the trade normalization always makes, and it is almost always worth it.

SLIDE 10 · YOUR TURN

The method, in four steps

1. Write down the functional dependencies, or confirm you can read the given ones aloud as sentences about the mini-world.
2. Find the candidate key by attribute closure, then check that no proper subset is also a key.
3. Name the violation precisely: non-atomic attribute, *partial* dependency, or *transitive* dependency.
4. Decompose, then say which anomaly you just made impossible.

> **FIVE EXERCISES**
>
> Two more on 1NF — one of which is a trap — then two on 2NF and one on 3NF. Exercises 4 and 5 are a chain over the same mini-world, so the second builds on the first.

HW3 is due Monday, November 9. Everything in these exercises is directly on it.